ABS with PYNQ Z1

Mattia Bortolaso VR500026

May 19, 2025

1 Summary

The objective of this project was to design and implement an Anti-lock Braking System (ABS) capable of running on the `PYNQ-Z1` development board, with a simplified approach that does not account for the mechanical delays associated with electro-valve and pump actuation times. The project involved developing the core logic required to simulate `ABS` behavior, integrating it with the `PYNQ-Z1` platform, and performing extensive simulations to assess system performance. Special attention was given to evaluating response time and verifying correct state transitions and functionality under a variety of simulated braking conditions.

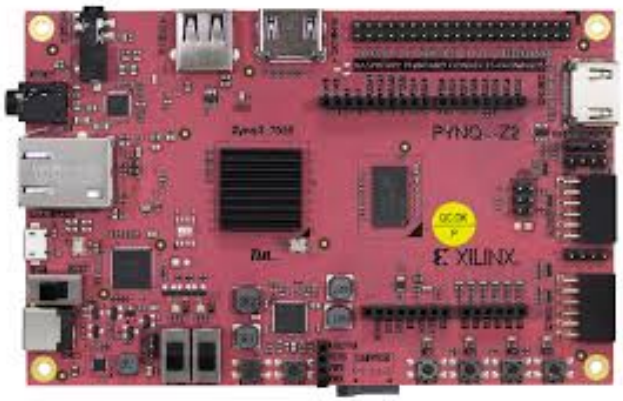

Figure 1: PYNQ-Z1 Board

The code for this project was written entirely in Verilog, specifically handwritten from scratch rather than generated from a C file. To simulate and test the `ABS` design without using the physical board, we utilized the Vivado software. Within Vivado, we performed behavioral simulations to verify the functionality of the system, as well as synthesis processes to test the design's feasibility and prepare it for implementation on the hardware.

2 Concept Used

2.1 EFSM (Extended Finite State Machine)

The first step in the project was to define the concept of an Extended Finite State Machine (EFSM) on paper. The `EFSM` was modeled with four distinct states to represent the key phases of `ABS` behavior:

- **NORMAL OPERATION:** The vehicle is functioning under normal conditions, with no signs of wheel lock-up.
- **ANTILOCKING:** A potential wheel lock-up has been detected based on predefined logic.
- **RELEASE PRESSURE:** In response to the detected lock-up, the system reduces the brake pressure on the affected wheel to regain traction.
- **REAPPLY PRESSURE:** Once the wheel is no longer locking, normal brake pressure is reapplied to maintain effective braking performance.

This `EFSM` framework formed the foundation for implementing the `ABS` logic in Verilog.

Current State	Next State	Condition
NORMAL_OPERATION	ANTILOCKING	`engine_status && vehicle_speed > 5 && slip_pct > SLIP_ENTER`
ANTILOCKING	NORMAL_OPERATION	`!brake_signal`
ANTILOCKING	RELEASE_PRESSURE	`antislip_counter >= MIN_ANTILOCKING_TIME && slip_pct < SLIP_EXIT`
RELEASE_PRESSURE	NORMAL_OPERATION	`!brake_signal`
RELEASE_PRESSURE	ANTILOCKING	`slip_pct > SLIP_ENTER || release_counter > MAX_RELEASE_TIME`
RELEASE_PRESSURE	REAPPLY_PRESSURE	`brake_signal && slip_pct <= SLIP_ENTER && release_counter <= MAX_RELEASE_TIME`
REAPPLY_PRESSURE	NORMAL_OPERATION	`!brake_signal || slip_pct < SLIP_EXIT`
REAPPLY_PRESSURE	ANTILOCKING	`slip_pct > SLIP_ENTER`
REAPPLY_PRESSURE	REAPPLY_PRESSURE	otherwise (stay in same state)

Table 1: EFSM Transition Table for the ABS System

2.2 How ABS Works

The ABS (Anti-lock Braking System) operates by continuously monitoring both the wheel speed and the overall vehicle speed. Using these inputs, the system calculates the slip ratio (*slip percentage*), based on known physical formulas. When the slip exceeds a predefined threshold—signaling a potential wheel lock-up—the system initiates corrective measures to maintain traction and avoid skidding.

Upon detecting a lock-up condition, the system performs the following actions:

1. **Closure of the Vrc1 electro-valve:** This prevents the driver from increasing brake pressure on the affected wheel, effectively isolating the wheel from additional hydraulic pressure input.

2. **Opening of the Vrc2 electro-valve:** This valve, which remains closed under normal braking conditions, is opened to recover brake fluid from the main caliber circuit. The reduction in pressure on the wheel helps to reduce the braking power and restore the wheels rotation.

3. **Brake fluid handling:** In a conventional ABS design, the recovered fluid would be temporarily stored in an expansion vessel located near the Vrc2 valve. However, for simplification purposes in this implementation, the expansion vessel is omitted. Instead, the system directly activates a recovery pump, which reroutes the brake fluid back into the main hydraulic circuit over the Vrc1, just after the brake pedal. This ensures that brake pressure could be restored promptly once the slip condition is resolved.

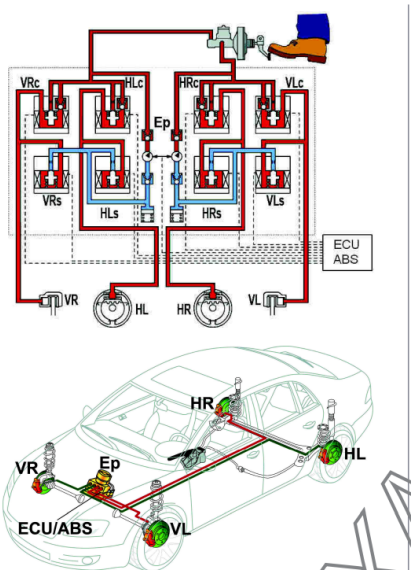

Figure 2: Schematic of valve behavior and brake fluid flow during ABS operation.

2.3 Slip Calculation

To calculate slip, it's important first to understand the concept of slip ratio and how to utilize it in the `ABS` system. The formula we employed to calculate the slip ratio is as follows:

$$\text{Slip} = \frac{(V_{\text{vehicle}} - V_{\text{wheel}}) \times 100}{V_{\text{vehicle}}}$$

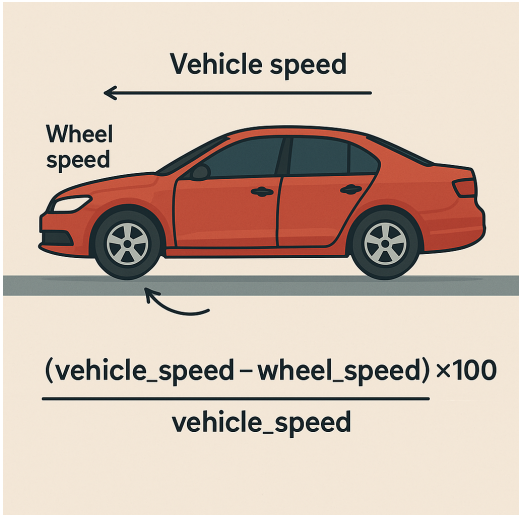

Figure 3: Visual explanation of the Slip formula.

In this formula, the speed of the wheel is subtracted from the vehicle speed, then multiplied by 100. The result is then divided by the vehicle speed to express the slip as a percentage.

One key challenge in calculating slip is accounting for the vehicle's turning dynamics. When the vehicle is turning left or right, the slip ratio can vary significantly depending on which tire the `ABS` system is monitoring. For instance, when turning right, the difference between the wheel speed and vehicle speed can exceed 40%. This variation necessitates adjusting the tolerance level for slip based on the turning direction and the specific tire the system is operating on. Therefore, the system must incorporate more flexible tolerance thresholds when the car is turning, depending on the wheel in use.

This adjustment ensures more accurate detection of wheel lock-up, considering the natural differences in wheel speeds while turning.

In the implementation of the ABS logic, we introduced two threshold variables for slip detection: `SLIP_ENTER = 20%` and `SLIP_EXIT = 10%`. These thresholds are used within conditional statements in the code to control transitions into and out of the `ANTILOCKING` state of the `EFSM`.

Specifically:

The system enters the `ANTILOCKING` state when the slip exceeds the `SLIP_ENTER` threshold.

It exits the `ANTILOCKING` state once the slip drops below the `SLIP_EXIT` threshold.

In the initial version of the code, we used only a single slip threshold for both entering and exiting the state. This resulted in the `EFSM` remaining in the `ANTILOCKING` state for too long, even after the slip condition had improved. By introducing separate thresholds for entry and exit, we created a hysteresis effect that prevents rapid toggling between states and allows for more stable and realistic `ABS` behavior.

2.4 Temporization of Anti-Locking State

In the third version of the code, I introduced an `anti_slip_counter`, a 3-bit counter designed to implement a minimum residence time in the `ANTILOCKING` state. Alongside this, I defined a constant `MIN_ANTILOCKING_TIME` = 3, which represents the minimum number of clock cycles (or time units, depending on system timing) that the system must remain in the `ANTILOCKING` state before transitioning out.

This mechanism ensures that once a wheel lock-up is detected and the system enters the `ANTILOCKING` state, it cannot exit immediately—even if the slip ratio falls below the `SLIP_EXIT` threshold. This adds robustness to the system by avoiding premature transitions caused by brief or noisy changes in slip values.

The `anti_slip_counter` is reset upon entering the `ANTILOCKING` state and incremented on each clock cycle while the system remains in that state. Only after the counter reaches `MIN_ANTILOCKING_TIME` does the system become eligible to transition out of `ANTILOCKING`, provided the slip condition is also met.

2.5 Transition Conditions

The state transitions in the `EFSM` are governed by straightforward conditional statements based on the slip percentage (`slip_pct`), vehicle speed, and driver input:

- When the system detects a significant difference between the vehicle speed and the wheel speed (indicated by the slip percentage), and the vehicle is moving while the driver is applying the brakes, the state transitions from **NORMAL_OPERATION** → **ANTILOCKING**.
- If the wheel remains locked (high slip), the system transitions from **ANTILOCKING** → **RELEASE_PRESSURE** to reduce brake pressure and prevent wheel lock-up.
- Once the wheel is no longer locked (slip has decreased below a threshold), the state transitions from **RELEASE_PRESSURE** → **REAPPLY_PRESSURE**, where brake pressure is gradually reapplied to maintain optimal braking.

2.6 Testbench

The goal of the testbench was to validate the `ABS` behavior under a variety of complex and realistic driving scenarios. Specifically, we designed test cases to simulate and manipulate the following input conditions:

- **Vehicle speed**
- **Individual wheel speeds** (for each wheel)
- **Brake signal** (driver brake input)
- **Engine status** (0 OFF, 1 ON)
- **Steering direction** (left, right, or straight)

By varying these inputs, we were able to emulate both normal and edge-case conditions, such as sudden braking during a turn or uneven traction across wheels.

The output signals analyzed during the testbench simulations included:

- **Slip offset:** used to track the difference between vehicle and wheel speed.
- **Vrc1 and Vrc2:** control signals related to brake pressure release or recovery.
- **Recovery pump signal:** indicates if and when the system attempts to restore pressure to the affected wheel.

This comprehensive testbench allowed us to evaluate the robustness and responsiveness of the `ABS` logic under different conditions, ensuring it behaves as expected across a wide range of driving situations.

3 Program Logic

3.1 Verilog Modules

The `ABS` system is composed of three Verilog files, each responsible for a specific function to ensure correct and coordinated operation:

- **ABS.v**
  This is the main module that contains the core ABS algorithm and the Extended Finite State Machine (EFSM). It handles all state transitions—such as entering and exiting the `ANTILOCKING` state—and implements the logic for monitoring slip and controlling brake pressure.
- **turn_detector.v**
  This module is responsible for detecting the steering direction of the vehicle. It determines whether the car is turning left or right and sets a corresponding direction output signal. This directional information is essential for adapting the ABS behavior based on the dynamics of turning.
- **direction_compensator.v**
  This module adjusts the `slip_offset` value based on the steering direction. When the steering wheel is straight, the `slip_offset` is set to 0. However, during turns, the module increases the `slip_offset` depending on whether the vehicle is turning left or right. This compensation allows the `ABS` system to tolerate higher slip ratios during normal turning, improving accuracy and preventing false lock-up detection.

4 Simulation

To ensure comprehensive validation of the `ABS` system, I developed and used two different versions of the testbench, each targeting specific braking scenarios.

In the first testbench (Testbench_ABS_V1.v), the focus was on evaluating the `ABS` behavior under basic and moderately complex conditions. This version simulated two primary cases:

Braking with no steering: The vehicle is moving straight, and the `ABS` logic is expected to react only to changes in wheel slip without any directional compensation.

Braking while steering to the right: In this scenario, the testbench simulates a right turn during braking. This allows us to verify the integration between the turn_detector and direction_compensator modules, ensuring that the system correctly adjusts the slip_offset and tolerates higher slip values on the appropriate wheels.

These scenarios allowed for effective validation of the basic `EFSM` transitions, slip detection, and the initial interaction of directional logic with the core `ABS` system.

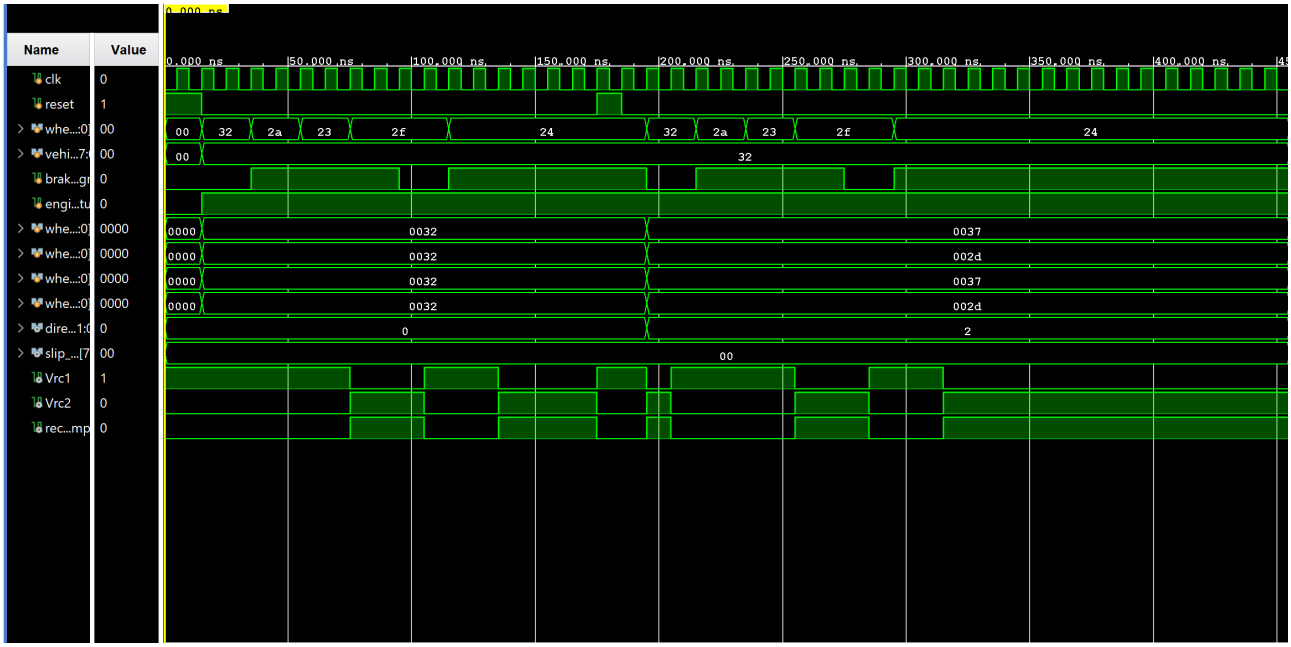

Figure 4: ABS Simulation Test – V1

The outputs obtained from the first simulation were fully consistent with the expected behavior of a typical `ABS` system under test conditions. Based on these promising results, I proceeded to run a second testbench, `Testbench_ABS_V2.v`, which provided a new set of input stimuli.

This version focused on testing the `ABS` response during braking while the vehicle was steering to the left, allowing further validation of the system's ability to adapt slip tolerance based on turning direction.

These additional tests helped confirm the robustness and adaptability of the implemented `ABS` logic under varied dynamic scenarios.

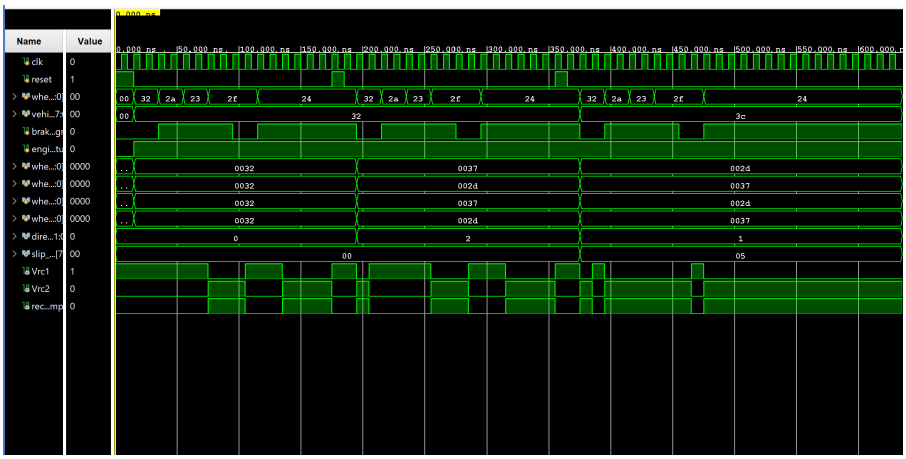

Figure 5: ABS Simulation Test – V2

5 Conclusions

The main objective of this project was successfully achieved, and the simulations conducted clearly demonstrated the correct and expected operation of the ABS system.

References

[1] TEXA EDU, *Introduzione ai sistemi ABS*, Dispense didattiche, 2024.

[2] Corso FPGA – Università di Verona, *Prof. Luigi Capogrosso*, Anno accademico 2024/2025.

[3] Gianluca Bortolaso, *Supporto e chiarimenti sul funzionamento dell'ABS*, discussioni personali, 2025.

[4] M. Bortolaso, *ABS-PYNQ-Z1 Project*, GitHub repository, 2025.
`https://github.com/Borto71/ABS-with-PYNQ-Z1`